**1. Detección de Necesidades**

- **Actividad realizada:** Se llevó a cabo un levantamiento de requerimientos y un análisis operativo profundo de los flujos de información, canalizaciones (*pipelines*) y métricas de salud dentro de la infraestructura tecnológica de PluriOne S.A. de C.V. Se detectó que la gestión y el monitoreo operaban bajo un esquema predominantemente reactivo ante cuellos de botella por latencia, anomalías en la calidad de los datos y fallos en las migraciones, lo que dificultaba la toma de decisiones directivas y técnicos. Con base en esto, se diagnosticó la necesidad crítica de implementar una plataforma centralizada y automatizada de observabilidad con capacidades de inteligencia artificial.
- **Entregable formal:** Documento de Requerimientos del Producto (PRD), el cual establece formalmente los objetivos, el alcance, los requerimientos funcionales y no funcionales, y la delimitación técnica del sistema.

**2. Propuesta**

- **Actividad realizada:** Se diseñó y presentó la propuesta técnica estratégica para la solución de observabilidad, determinando la viabilidad operativa y económica de integrar una arquitectura moderna orientada a microservicios. Se definió la estrategia de monitoreo en tiempo real mediante WebSockets, la persistencia estructurada de bitácoras de incidentes y la incorporación de un asistente de diagnóstico inteligente.
- **Entregable formal:** Propuesta técnica de la solución, integrada en el presente documento, la cual detalla la justificación del *stack* tecnológico seleccionado (FastAPI, React.js, PostgreSQL y Docker) y su alineación directa con los objetivos de optimización de DataOps de la empresa.

**3. Arquitectura**

- **Actividad realizada:** Se estructuró el diseño de la arquitectura de software y los diagramas de integración de componentes. Se definieron las interfaces de programación de aplicaciones (APIs RESTful) gestionadas por FastAPI, los canales de comunicación bidireccional mediante WebSockets, el modelo

relacional para la base de datos en PostgreSQL y la estrategia de despliegue contenerizado utilizando Docker y Docker Compose para garantizar la portabilidad y replicabilidad del entorno en cualquier servidor.

- **Entregable formal:** Documento de Arquitectura y Diseño de Sistemas, el cual plasma la interacción síncrona y asíncrona entre el cliente web (React.js), el servidor de procesamiento y la base de datos corporativa.

**4. Planificación**

- **Actividad realizada:** Se estableció la planeación operativa y la gestión metodológica del proyecto bajo un marco de trabajo ágil (*Scrum*), organizando el desarrollo en ciclos iterativos (Sprints) para garantizar entregas funcionales y controladas.
- **Entregable formal:** Plan del Proyecto, Cronograma General y Backlog de Desarrollo detallado de la siguiente manera:
    - **Cronograma General de Fases:**
        - *Fase 1 a 4 (Análisis, Propuesta, Arquitectura y Planificación):* Semanas 1 y 2.
        - *Fase 5 (Desarrollo del Sistema Core, Backend, Frontend e IA):* Semanas 3 a 6.
        - *Fase 6 a 10 (Pruebas QA, Documentación, Mantenimiento, Capacitación y Cierre):* Semanas 7 y 8.
    - **Backlog Detallado de Sprints:**
        - **Sprint 1 (Fundamentos y Base de Datos):** Configuración del entorno de desarrollo, creación de la estructura base del repositorio e implementación del esquema relacional en PostgreSQL para el almacenamiento seguro de usuarios, pipelines y registros de anomalías.

- **Sprint 2 (Backend, API y Seguridad):** Desarrollo de la API REST utilizando Python y FastAPI, establecimiento de los endpoints para la gestión de datos, configuración de seguridad basada en autenticación por tokens (JWT) e implementación de la lógica de WebSockets para notificaciones en tiempo real.
- **Sprint 3 (Frontend, Visualización e Inteligencia Artificial):** Construcción de la interfaz de usuario moderna en React.js con diseño responsivo, integración de componentes gráficos (Recharts) para el monitoreo de métricas (latencia, tasas de error y calidad) y habilitación del panel conversacional del asistente inteligente de diagnóstico (J.A.R.V.I.S.).
- **Sprint 4 (Contenedorización, QA y Cierre):** Empaquetamiento de todo el ecosistema mediante Docker Compose para su ejecución unificada, ejecución de pruebas de integración y elaboración de los manuales y entregables finales.

## 5. Desarrollo

- **Actividad realizada:** Se llevó a cabo la implementación integral del código fuente y la construcción de los componentes core del sistema, materializando la plataforma de observabilidad de datos con inteligencia artificial para PluriOne S.A. de C.V. El proceso se estructuró en dos capas principales:
  - **Backend y Lógica de Negocio (Python / FastAPI):** Se programaron los endpoints de la API REST para el manejo de sesiones de usuario, consultas de bitácoras y gestión de pipelines. Se implementó el sistema de seguridad basado en *JSON Web Tokens* (JWT) para la

autenticación y encriptación de credenciales. Asimismo, se integró el motor de comunicación asíncrona mediante **WebSockets** para transmitir de manera instantánea las alertas de anomalías hacia los clientes conectados. Se configuró la persistencia relacional conectando el servidor con **PostgreSQL**, permitiendo almacenar de forma estructurada los eventos de latencia, calidad de datos y porcentajes de fallo.

    - **Interfaz de Usuario y Visualización (React.js):** Se desarrolló un panel de control (*dashboard*) interactivo y responsivo utilizando React.js y librerías de componentes gráficos (Recharts) para renderizar en tiempo real el comportamiento de los pipelines críticos de la empresa. Se incorporó el módulo de consultas inteligentes asistido por el agente **J.A.R.V.I.S.**, permitiendo a los operadores obtener diagnósticos automatizados y recomendaciones de causa raíz ante incidentes críticos.

- **Entregable formal:** Sistema Funcional, representado por el código fuente completo del proyecto alojado en el repositorio corporativo, estructurado modularmente con sus directorios de backend/, frontend/, configuraciones de base de datos y el archivo de orquestación docker-compose.yml que permite levantar todo el ecosistema de manera unificada.

**6. Pruebas**

- **Actividad realizada:** Se ejecutó un ciclo riguroso de pruebas de calidad (*Quality Assurance - QA*) y validación técnica sobre la plataforma de observabilidad para garantizar la estabilidad, seguridad y el rendimiento óptimo del sistema antes de su entrega corporativa. El proceso abarcó las siguientes dimensiones:

    - **Pruebas Funcionales y de Integración:** Se validó la correcta comunicación entre el frontend desarrollado en React.js y el backend en FastAPI, comprobando que las peticiones HTTP y los eventos

transmitidos por WebSockets operaran sin pérdida de paquetes ni latencia perceptible. Se verificó el almacenamiento y la recuperación íntegra de los registros de anomalías en la base de datos PostgreSQL.

- **Pruebas de Seguridad y Autenticación:** Se evaluaron los mecanismos de control de acceso, comprobando que la generación, validación y expiración de los *JSON Web Tokens* (JWT) restrinjan adecuadamente las rutas protegidas del sistema y que las credenciales se almacenen bajo estrictos estándares de cifrado (*hashing*).
- **Pruebas de Rendimiento y Disponibilidad:** Se simuló la inyección masiva de eventos y alertas en los pipelines monitoreados para medir el tiempo de respuesta del servidor y la fluidez en el renderizado de gráficos dinámicos, asegurando alta disponibilidad bajo entornos contenerizados con Docker.

- **Entregable formal:** Reporte QA (Informe técnico de validación de calidad, el cual documenta la superación exitosa de las pruebas funcionales, de integración, de seguridad y de rendimiento del software).

## 7. Documentación

- **Actividad realizada:** Se elaboró la documentación formal del sistema, estructurando el manual técnico y de usuario para garantizar la transferencia tecnológica, la correcta operación y la facilidad de mantenimiento de la plataforma de observabilidad en PluriOne S.A. de C.V.
- **Entregable corporativo (Manual Técnico y de Usuario):** El contenido integrado en esta sección y respaldado en el repositorio del proyecto comprende los siguientes apartados:

### 1. Guía de Instalación y Configuración del Entorno:

- Requisitos previos de software y dependencias (Python, Node.js, Docker y Docker Compose).
- Configuración de variables de entorno (archivo .env para la conexión a PostgreSQL, claves de seguridad JWT y servicios externos).
- Instrucciones paso a paso para la ejecución automatizada mediante contenedores (docker-compose up --build).

**2. Documentación de la API y Componentes del Sistema:**

- Estructura de directorios del repositorio (backend/ con FastAPI y frontend/ con React.js).
- Descripción técnica de los endpoints principales para el registro y consulta de pipelines y métricas de salud.
- Mecanismo de autenticación y seguridad basado en *JSON Web Tokens* (JWT).

**3. Manual de Operación para el Usuario:**

- Guía de acceso y navegación en el panel de control (*dashboard*).
- Interpretación visual de las alertas en tiempo real y gráficos de latencia o tasa de errores.
- Modo de uso e interacción con el asistente de diagnóstico inteligente de causa raíz (J.A.R.V.I.S.).

**8. Mantenimiento**

- **Actividad realizada:** Se diseñó e implementó el esquema de soporte técnico, mantenimiento correctivo/preventivo y mejora continua para garantizar la continuidad operativa, la estabilidad y la alta disponibilidad de la plataforma de observabilidad en PluriOne S.A. de C.V.

- **Entregable corporativo (Plan de Mantenimiento):** Se estructuró un plan de soporte multinivel y gestión de actualizaciones integrado por los siguientes componentes:
    - **Soporte Nivel 1 (Monitoreo Preventivo y de Salud):** Supervisión periódica de la disponibilidad de los contenedores mediante Docker, verificación del espacio en disco para el almacenamiento de registros y métricas en PostgreSQL, y control de los tiempos de respuesta de los servicios en FastAPI.
    - **Soporte Nivel 2 (Diagnóstico y Corrección de Incidentes):** Protocolo para la depuración de errores lógicos en los flujos de datos, revisión de logs del sistema ante fallos de conexión o autenticación (JWT), y actualización de librerías y dependencias en el ecosistema de Python y Node.js.
    - **Estrategia de Mejora Continua y Mantenimiento Evolutivo:** Lineamientos técnicos para la incorporación progresiva de nuevos pipelines de datos, optimización de los componentes visuales en React.js y calibración de los algoritmos de análisis del asistente inteligente de diagnóstico (J.A.R.V.I.S.).

**9. Capacitación**

- **Actividad realizada:** Se estructuró el programa de transferencia de conocimientos y habilitación técnica dirigido a los administradores de sistemas, ingenieros de datos (*Data Engineers*) y operadores de PluriOne S.A. de C.V., con el propósito de asegurar la adopción correcta y el aprovechamiento óptimo de la plataforma de observabilidad.
- **Entregable corporativo (Acta de Capacitación):** Se documentó formalmente el protocolo de instrucción y entrega técnica, el cual comprende:
    - **Instrucción a Administradores:** Guía sobre la gestión de contenedores en Docker, supervisión de bases de datos en PostgreSQL y control de accesos mediante seguridad JWT.

- **Instrucción a Data Engineers y Operadores:** Demostración del uso de los dashboards interactivos en React.js, interpretación de las métricas de latencia y calidad de datos, y seguimiento de alertas en tiempo real.
- **Adopción del Asistente Inteligente:** Capacitación sobre la interacción con el módulo de diagnóstico automatizado de causa raíz (J.A.R.V.I.S.).
- **Formalización:** Emisión del acta de instrucción técnica que certifica que el personal cuenta con las bases operativas para el uso del sistema.

**10. Cierre**

- **Actividad realizada:** Se llevó a cabo la evaluación integral de los resultados obtenidos frente a los objetivos iniciales planteados para el proyecto en PluriOne S.A. de C.V., validando el cumplimiento de la propuesta técnica, la estabilidad del sistema y la correcta ejecución de cada fase del desarrollo.
- **Entregable corporativo (Informe Final):** Se consolidó el balance general del proyecto, documentando los siguientes apartados clave:
  - **Evaluación de Resultados:** Verificación del cumplimiento en la reducción de tiempos de detección y resolución de incidentes en los flujos de información, gracias a la implementación de la plataforma automatizada de observabilidad basada en WebSockets e inteligencia artificial.
  - **Lecciones Aprendidas:** Análisis técnico de los retos superados durante la integración del *stack* tecnológico (Python, FastAPI, React.js, PostgreSQL y Docker), así como la importancia de una arquitectura modular y escalable para el monitoreo de datos empresariales.

- **Conclusión y Entrega Técnica:** Declaración formal del cierre del proyecto, entregando el sistema plenamente funcional, documentado y listo para su operación continua dentro de los servicios tecnológicos de Develop Talent & Technology (PluriOne).